Challenge 48h

Parkshare : Étude de marché par la donnée

Dev	Data	Infra

Contexte

Parkshare est une plateforme de partage de places de stationnement entre résidents d'une même copropriété.

L'entreprise cherche à identifier les zones géographiques à fort potentiel commercial pour cibler ses actions de prospection auprès des syndics et bailleurs.

Votre mission : construire l'outil d'analyse qui rendra cette prospection possible.

Site web : https://parkshare.net/fr/

LinkedIn : https://www.linkedin.com/company/parkshare-net

Organisation générale

Structure du projet

Le projet est réparti entre trois profils complémentaires qui travaillent en pipeline :

Profil	Responsabilité principale	Dépendances
Data	Collecte, nettoyage, analyse, production des KPIs	Aucune dépendance entrante, livre en premier
Dev	Base de données, dashboard interactif	Reçoit les données de Data
Infra	Containerisation, déploiement, sécurisation	Déploie ce que Dev et Data ont produit

Recommandation : Commencez à chercher les données et choisissez un axe d'orientation ensemble, puis n'attendez pas que la personne en Data finisse de traiter les données. Configurer le projet, le repo git, l'API, le serveur, etc…

Structure du repo Git

Un seul repo partagé entre les trois profils, avec la structure suivante :

Structure du repo

```
/data        → notebooks, scripts de collecte et de nettoyage, fichiers de
livraison
/app         → code du dashboard, schéma de base de données
/infra       → Dockerfile, docker-compose.yml, config reverse proxy
.env.example → variables d'environnement nécessaires (sans valeurs réelles)
README.md    → point d'entrée global, liens vers les README de chaque
dossier
```

Partie Data

Objectif principal

Collecter, nettoyer et analyser des données ouvertes pour produire des KPIs d'analyse dans le but de fournir une étude des zones à fort potentiel commercial pour Parkshare.

Données d'entrée

Chaque groupe devra trouver ses propres jeux de données. Cela peut aller du registre des copropriétés en France, au taux de motorisation dans certaines communes, en passant par le taux de stationnement privé dans les grandes villes. Vous êtes libre de choisir.

Il est nécessaire de trouver des données géographiques dans le but de réaliser une carte sur le dashboard réalisé par la partie Dev. Cela peut être des coordonnées, des zones, etc...

Exemple de sources de données

- **Données INSEE**
  - Densité de population, taux de motorisation par commune
  - Part de logements collectifs vs individuels, taille moyenne des ménages
- **Base DVF / data.gouv.fr**
  - Transactions immobilières pour identifier les copropriétés avec lots de stationnement
- **Open data communale**
  - Nombre de copropriétés, zonage résidentiel
- **Toute autre source pertinente de votre choix**

⚠ Éviter les sources nécessitant du scraping intensif (Google Places, Pages Jaunes) : le rapport coût/bénéfice en 48h est défavorable. Si vous souhaitez explorer une telle source, cantonnez-la à un bonus et n'en faites pas un point critique du pipeline.

KPIs minimum attendus

KPI	Description
Score de potentiel	Par ville ou arrondissement, croisant les données exploités. Méthode de scoring et pondérations au choix de l'équipe.
Classement des zones	Top N zones prioritaires pour la prospection commerciale de Parkshare.

+ au moins deux autres KPIs de votre choix

Rendu attendu

- Un fichier Jupyter Notebook (.ipynb), script python ou équivalent comprenant l'ensemble des développements
- Le fichier de livraison de données
- Un README.md dans /data/ décrivant : les sources utilisées, les instructions pour reproduire le pipeline, l'explication des graphiques et des KPIs

Indications techniques

- Traitement de données : Pandas, Polars, Spark ou tout autre outil adapté
- Construction des KPIs / graphiques : Matplotlib, Seaborn, Plotly ou tout autre outil
- Gestion des dépendances : requirements.txt ou environment.yml

Partie Dev

Objectif principal

Structurer et stocker l'ensemble des données produites par l'équipe Data dans une base de données avec un schéma documenté, puis construire par-dessus un dashboard interactif connecté à cette base pour offrir un rendu visuel et explorable des résultats de l'étude de marché.

Base de données

La base de données doit comprendre au minimum trois types de tables :

KPI	Description
Tables sources brutes	Données telles que reçues de l'équipe Data, sans transformation. Permettent la traçabilité et la reproductibilité.
Tables transformées	Données nettoyées, enrichies, jointures effectuées. C'est sur ces tables que les KPIs sont calculés.
Tables de KPIs	Résultats agrégés prêts à l'affichage : scores par zone, classements, indicateurs clés.

Le schéma de la base de données doit être documenté (diagramme ou description textuelle des tables, colonnes, types et relations).

Fonctionnalités du dashboard

- **Une carte interactive affichant le scoring**
- **Des filtres dynamiques**
- **Des graphiques dynamiques sur les KPIs**

Rendu attendu

- Une base de données peuplée avec les données de l'équipe Data
- Une application dashboard fonctionnelle connectée à cette base
- Les fichiers de code versionné dans /app/
- Un README.md dans /app/ décrivant : le schéma de la base, les instructions de connexion et d'utilisation du dashboard

Indications techniques

- Base de données : relationnelle (PostgreSQL, SQLite) ou analytique (DuckDB) selon votre cas d'usage
- Dashboard : Streamlit, Dash, ou application web custom (React, Vue, FastAPI...)
- Cartographie : Folium, Plotly Mapbox, Leaflet.js ou équivalent

⚠ Éviter les outils de BI no-code (Metabase, Grafana) : ce challenge attend du développement, pas de la configuration.

Partie Infra

Objectif principal

Conteneuriser et déployer la stack (base de données, dashboard), rendre les services accessibles à distance aux autres membres de l'équipe, et sécuriser les accès de manière à ce que seuls les services nécessaires soient exposés.

Containerisation

Les services de la stack doivent être conteneurisé indépendamment et orchestré via Docker Compose ou autre orchestrateur :

- Base de données (PostgreSQL, SQLite via service dédié, ou Duck DB selon le choix Dev)
- Application dashboard (équipe Dev)
- Reverse proxy (Nginx ou Traefik)

Le fichier docker-compose.yml doit être à la racine de /infra/ et permettre de démarrer la stack avec une seule commande.

Déploiement à distance

Les services doivent être accessibles depuis une machine extérieure au réseau local de l'équipe.

- Hébergement : VPS, cloud provider, ou tout environnement distant accessible
- L'URL d'accès doit être documentée dans le README infra et testée depuis une machine externe
- HTTPS recommandé (Let's Encrypt via Certbot ou Traefik)

Sécurité

Deux règles essentielles :

- Gérer les secrets via variables d'environnement. Versionner un fichier .env.example dans le repo avec les noms des variables mais sans leurs valeurs.
- Limiter ce qui est exposé : seuls le reverse proxy et le dashboard ont vocation à être accessibles depuis l'extérieur. La base de données reste dans le réseau interne Docker.

Rendu attendu

- Les fichiers de configuration de déploiement (docker-compose.yml, Dockerfile(s), config reverse proxy)
- Le fichier .env.example versionné
- Les services accessibles à distance et vérifiés depuis une machine externe
- Un README.md dans /infra/ décrivant : comment déployer, comment accéder aux services, les variables d'environnement nécessaires

Indications techniques

- Containerisation : Docker, Docker Compose
- Reverse proxy : Nginx, Traefik
- Hébergement : VPS OVH, Scaleway, DigitalOcean, AWS/GCP/Azure free tier, ou tout environnement distant
- HTTPS : Let's Encrypt (Certbot) ou Traefik avec résolution automatique
- Secrets : variables d'environnement, jamais en dur dans le code

Où se font les rendus ?

- Sur MOODLE
  - La présentation finale
  - Le lien vers GIT (public)
- Sur GIT
  L'ensemble des rendus techniques attendus

Critères d'évaluation

Critères communs aux trois profils

- Qualité et clarté du code (lisibilité, structure, commentaires)
- Qualité de la documentation (README, schémas)
- Coopération inter-profils : fluidité de la transmission entre équipes, réactivité face aux blocages, adaptation si une dépendance prend du retard

Critères spécifiques Data

- Pertinence et diversité des sources exploitées
- Qualité du nettoyage et de la transformation des données
- Justesse et clarté des KPIs produits
- Reproductibilité du pipeline (un tiers peut relancer le notebook et obtenir les mêmes résultats)

Critères spécifiques Dev

- Schéma de base de données documenté et cohérent
- Dashboard fonctionnel avec carte interactive, filtres et graphiques dynamiques
- Qualité de l'expérience utilisateur du dashboard

Critères spécifiques Infra

- Stack complète containerisée
- Services accessibles depuis une machine extérieure au réseau local
- Absence de secrets dans le repo
- .env.example présent et complet
- HTTPS si services exposés (bonus si mis en place)

Bonus

Les bonus ne sont à traiter que si l'objectif principal est entièrement réalisé et documenté. Un objectif principal incomplet avec des bonus vaut moins qu'un objectif principal solide sans bonus.

Bonus 1 : Sources complémentaires (Data)

Enrichir l'analyse avec des sources additionnelles pour affiner le scoring :

- Données DRIEAT sur le stationnement résidentiel en Île-de-France
- Données PLU (Plan Local d'Urbanisme) pour identifier les zones résidentielles denses
- Portails open data des métropoles (Grand Lyon, Bordeaux Métropole, etc.)
- Toute source open data pertinente de votre choix, à justifier dans le README

  Données DRIEAT - PLU - Portails open data des métropoles – autres sources open data à justifier dans le README

Bonus 2 : Analyse de corrélation (Data + Dev)

Approfondir l'analyse en calculant les corrélations entre les différentes variables et en les visualisant dans le dashboard sous forme de nuage de points ou d'une heatmap de corrélation. L'objectif est de valider ou d'infirmer les hypothèses derrière le scoring.

Bonus 3 : Automatisation du pipeline (Data + Infra)

Mettre en place un scheduler (Airflow, Prefect, simple cron) pour automatiser la mise à jour des données et le recalcul des KPIs à intervalle régulier. Documenter le pipeline et le scheduler dans /infra/.

Bonus 4 : Accès à la finale

À l’issue de la présélection, les quatre équipes retenues auront accès à la finale et bénéficieront d’un bonus de points pour leur présentation devant le jury final

Présentation orale

Chaque groupe présente son travail à l'oral devant jury.

Structure suggérée

- Contexte et répartition du travail : qui a fait quoi, comment vous vous êtes coordonnés
- Démonstration du dashboard en live : navigation sur la carte, filtres, lecture des graphiques
- Choix techniques : sources de données retenues et pourquoi, méthode de scoring, stack technique
- Retour d'expérience : ce qui a bien fonctionné, ce qui a bloqué, ce que vous feriez différemment

Ce que le jury évalue à l'oral

- La capacité à expliquer vos choix techniques, pas seulement à les montrer
- La cohérence entre les trois parties : est-ce que l'ensemble tient comme un produit fini ?
- L'honnêteté sur les limites : un groupe qui identifie clairement ses compromis vaut mieux qu'un groupe qui survend un résultat fragile
- La fluidité de la démo : le dashboard doit tourner en live, pas en capture d'écran

Conseil : préparez un jeu de données stable pour la démo et testez le dashboard sur la machine de présentation.

Note finale

Ce challenge simule les conditions d'un projet data en entreprise : des dépendances entre profils, des délais contraints, et un livrable qui doit fonctionner devant un jury. Savoir coder ne suffit pas. Savoir se coordonner, livrer à temps et expliquer ses choix, c'est ce qui fait la différence.

Bonne chance.